



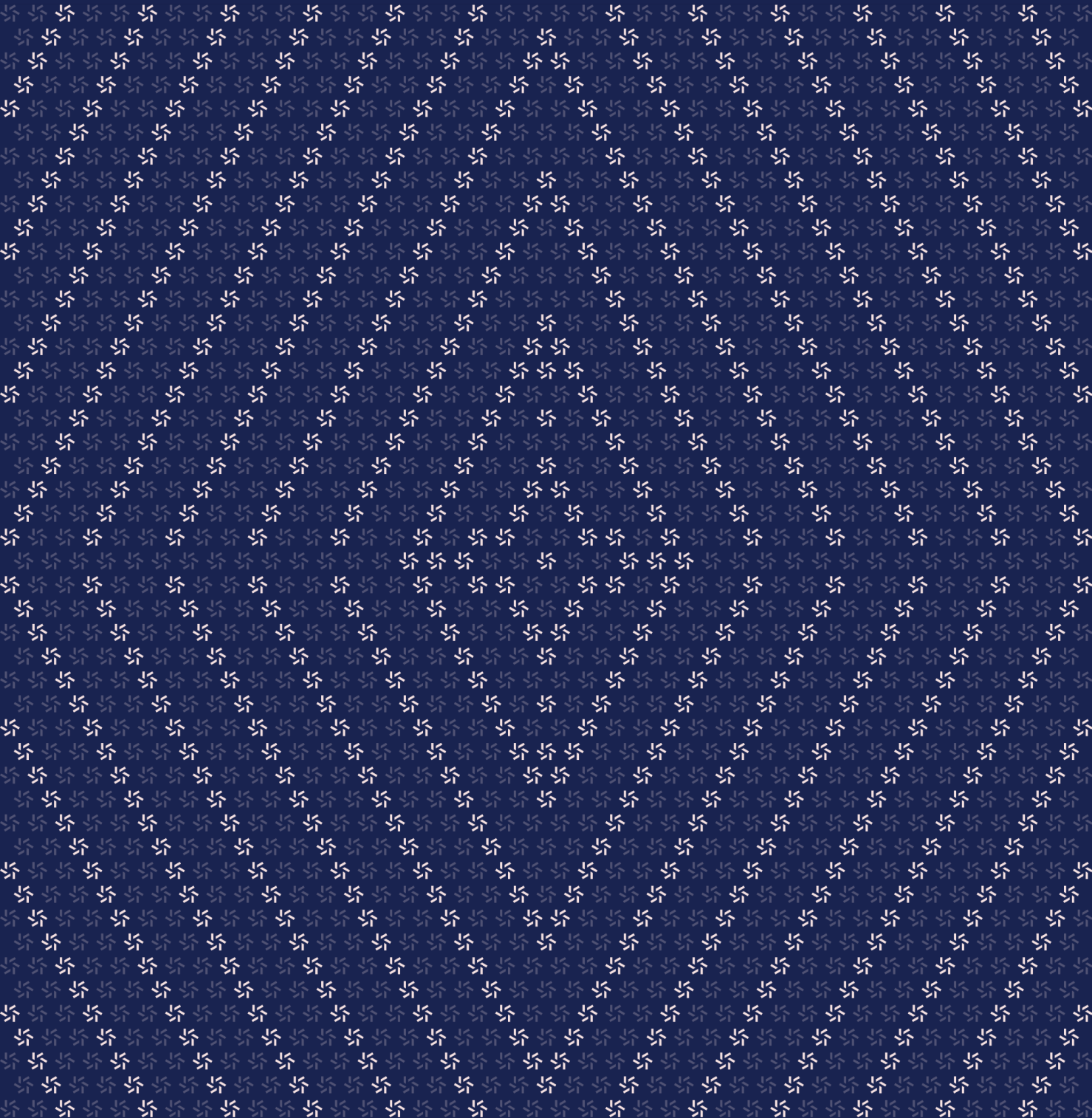
Prepared for
John Doherty
Nunchi

Prepared by
Filipe Alves
Kritsada Dechawattana
Ayaz Mammadov
Zellic

October 13, 2025

Yield Perps

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About Yield Perps	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Possibility of avoiding liquidation by opening orders to maintain equity	11
3.2. Canceling orders in loss scenarios can cause insufficient funds to pay users	14
3.3. Incorrect initialization of funding accrued for new positions	17
3.4. Liquidation denial of service via uint8 overflow in <code>Orderbook::getOpenOrders</code>	19
3.5. AMM denial of service when proxy orders match as takers	21
3.6. Mark price does not account for canceled orders	24
3.7. Inconsistency in funding-accrual calculation for <code>isLiquidatable</code>	26
3.8. Inconsistent funding accrual across <code>liquidateExt</code> and <code>placeOrder</code> functions	29

3.9.	Potential liquidation when matching same user orders	32
3.10.	Inaccurate AMM liquidity accounting	35
3.11.	Insufficient test-suite code coverage	37
3.12.	Insufficient treasury funds could prevent closing profitable positions	39
3.13.	No position-size validation allows unlimited collateral-free order creation	40
3.14.	Order cancellation can cause denial of service in matching	42
3.15.	Inflexible price-staleness check for diverse asset types	44
3.16.	Risk of repeatedly creating initial bins exceed the configuration limit	45
3.17.	Unsafe casting of negative rates causes underflow in <code>OracleManager::getUnderlyingPrice</code>	47
3.18.	Oracle staleness limit allows extended price delays	49
3.19.	Griefing attack via <code>updatePriceFeedsIfNecessary</code> function	51
3.20.	PythFeeds ignores confidence intervals, violating best practices	53
3.21.	Arbitrage and sandwiching	55
4.	Discussion	55
4.1.	Audit assumptions	56
5.	Threat Model	57
5.1.	Module: <code>Orderbook.sol</code>	58
6.	Assessment Results	63
6.1.	Disclaimer	64

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Nunchi from September 22st to October 2th, 2025. During this engagement, Zellic reviewed Yield Perps's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there potential attack vectors or manipulation risks in trade matching and profit and loss (PnL) calculations?
 - Is the vault architecture secure for storing and handling user funds?
 - Does the AMM design expose any avenues for fund loss or theft?
 - Are there mechanisms by which trades could be unfairly gamed or exploited?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Yield Perps contracts, we discovered 21 findings. Six critical issues were found. Nine were of high impact, three were of medium impact, and two were of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Nunchi in the Discussion section ([4.7](#)).

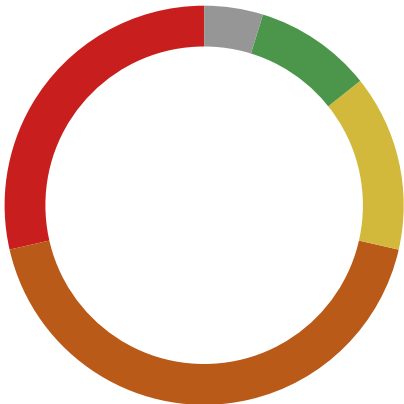
Based on the number of severe findings uncovered during the audit, it is our opinion that the

project is not yet ready for production. We strongly advise a comprehensive reassessment before deployment to help identify any potential issues or vulnerabilities introduced by necessary fixes or changes. We also recommend adopting a security-focused development workflow, including (but not limited to) augmenting the repository with comprehensive end-to-end tests that achieve 100% branch coverage using any common, maintainable testing framework, thoroughly documenting all function requirements, and training developers to have a security mindset while writing code.

The proposed remediations and fixes were thoroughly reviewed; however, many remained contentious due to the architectural and design decisions that underpin the protocol. Since several core functionalities are still being reworked, not all design choices were finalized during the remediation phase – even for issues that were technically addressed.

Breakdown of Finding Impacts

Impact Level	Count
Critical	6
High	9
Medium	3
Low	2
Informational	0
Indeterminate	1



2. Introduction

2.1. About Yield Perps

Nunchi contributed the following description of Yield Perps:

As billions in yield-bearing assets come on-chain, sophisticated participants lack the capital-efficient tools to hedge or trade this yield directly. Nunchi is building the institutional-grade perpetuals exchange for yield itself - turning complex rate exposures, from T-Bill yields and staking APRs to funding rates and volatility, into simple, liquid perpetuals. Liquidity is powered by a binned AMM, which feeds directly into the on-chain central limit order book (CLOB). This design allows users to provide liquidity, share in trading fees, and ensures that all orders and positions are transparently matched and settled on-chain.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Yield Perps Contracts

Type	Solidity
Platform	EVM-compatible
Target	contracts-core
Repository	https://github.com/Nunchi-trade/contracts-core
Version	62be384662b185b4abbcc72c4d0beb5b7eb86d70
Programs	Integrations/*.sol lib/*.sol Oracle/*.sol AMM.sol BinManager.sol Orderbook.sol PerpManager.sol Treasury.sol Vault.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.7 person-weeks. The assessment was conducted by three consultants over the course of two calendar weeks.

Contact Information

The following project managers were associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

Chad McDonald
✈ Engagement Manager
chad@zellic.io ↗

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
✈ Engineer
filipe@zellic.io ↗

Kritsada Dechawattana
✈ Engineer
kritsada@zellic.io ↗

Ayaz Mammadov
✈ Engineer
ayaz@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 22, 2025 Kick-off call

September 22, 2025 Start of primary review period

October 2, 2025 End of primary review period

3. Detailed Findings

3.1. Possibility of avoiding liquidation by opening orders to maintain equity

Target	Orderbook		
Category	Business Logic	Severity	Critical
Likelihood	High	Impact	Critical

Description

Position liquidation is determined by comparing equity against maintenance margin. However, the equity calculation is based on either `p.lockedCollateral` or `vault.lockedBalance(user)` depending on whether the user is AMM or not. This value increases whenever a user opens new orders. As a result, users whose positions are approaching liquidation can open new orders to increase their collateral, thereby boosting their equity to avoid liquidation.

The root cause of this issue is that the protocol treats the user's position and orders as the same collateral. This enables users to manipulate their position's collateral levels as needed.

```
function isLiquidatable(address user) public view returns (bool) {
    [...]
    Position memory p = positions[user];
    int256 fundingDue = (newCum - p.fundingAccrued) * p.size; // sign-aware

    // 3. Recompute equity including funding

    int256 uPnL = _unrealizedPnL(p, markPrice);

    uint256 lockedCollateral = p.isAMM ? p.lockedCollateral : vault.lockedBalance(user);
    int256 equity = int256(lockedCollateral) + p.realizedPnL - fundingDue + uPnL;

    // 4. Compare vs maintenance - use entry price for maintenance calculation
    // This ensures that as price drops, maintenance requirement doesn't drop proportionally
    uint256 entryPrice = 0;
    if (p.size != 0) {
        // Calculate absolute entry price from entryNotional and size
        entryPrice = uint256(_abs((p.entryNotional * 1e18) / p.size));
    } else {
        entryPrice = markPrice; // Fallback if no position
    }
}
```

```

uint256 maint = _maintenanceMarginReq(_abs(p.size), entryPrice);
return equity < int256(maint);
}

function placeOrder(
    bool isLong,
    uint128 price,
    uint128 amount,
    uint128 maxSlippage, // max slippage in price units
    uint16 leverage, // leverage multiplier (1-100x)
    address user,
    bytes calldata oracleData
) external nonReentrant onlyManager returns (uint256 id) {
    [...]
    uint256 collateralRequired = _marginReq(amount, uint256(price), leverage);
    if (10_000 / leverage < maintenanceMarginBps) {
        revert InsufficientMargin();
    }
    if (!isProxy) {
        vault.lock(user, collateralRequired);
    }

    positions[user].lockedCollateral += collateralRequired;
    // fix stack too deep
    id = _createOrder(isLong, price, amount, maxSlippage, leverage, user,
        isProxy);

    emit OrderPlaced(id, user, isLong, price, amount, collateralRequired,
        isProxy);

    _matchIncoming(id);
}

```

Impact

Users can avoid liquidation by simply opening new orders to boost their equity. If the liquidation process is not handled properly, the protocol may face insufficient funds to pay users who are entitled to profits.

Recommendations

Consider separating users' position collateral from their order collateral to ensure equity is calculated properly. This would prevent users from manipulating their equity to avoid liquidation.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [224d5627](#).

3.2. Canceling orders in loss scenarios can cause insufficient funds to pay users

Target	Orderbook		
Category	Business Logic	Severity	Critical
Likelihood	High	Impact	Critical

Description

The `liquidate` and `_closePosition` functions first cancel open orders to refund collateral back to the user then use the remaining collateral to cover any losses incurred. However, the equity calculation that determines the total amount available to pay losses is based on locked collateral, which represents the total collateral from open orders.

This creates situations where the losses are covered with less collateral than expected, potentially leaving insufficient funds to pay users who should receive profits.

```
function liquidate(address user, address caller) internal {
    Position storage p = positions[user];

    int256 size = p.size;
    if (size == 0) revert NoPosition();
    if (!isLiquidatable(user)) revert NotLiquidatable();
    _cancelOpenOrders(user);
    uint256 collateral = p.lockedCollateral;
    [...]
}
[...]
function _closePosition(address user) internal {
    _accrueFunding(user);

    Position storage p = positions[user];
    int256 size = p.size;

    _cancelOpenOrders(user);

    uint256 currentPrice = getMarkPrice();

    int256 unrealizedPnl = _unrealizedPnl(p, currentPrice);

    // we need to factor in pnl for the lifetime of the position
    int256 closingPnl = unrealizedPnl + p.realizedPnl;
```

```
// Calculate portion of locked collateral to release
uint256 collateralToRelease = p.lockedCollateral;
[...]
}
[...]
function cancelOrder(uint256 id, address user) public onlyManager {
    [...]
    // Calculate refund based on remaining unfilled amount in the order
    uint256 refund = _marginReq(uint256(remaining), uint256(order.price),
    order.leverage);
    // Update position's locked collateral
    Position storage p = positions[user];

    // we should always be able to cover refund
    assert(p.lockedCollateral >= refund);

    p.lockedCollateral -= refund;

    QueueWithVolume storage queueWithVolume = _queueWithVolume(order.isLong,
    order.price);
    if (queueWithVolume.volume < remaining) remaining
    = uint128(queueWithVolume.volume);
    queueWithVolume.volume -= remaining;
    if (!order.isProxy) {
        // unlock from vault
        vault.unlock(user, refund);
    } else {
        amm.onCancelOrder(order.trader, refund);
    }
    [...]
}
```

Impact

Refunding collateral before collecting the losses incurred may result in insufficient funds to pay the users who should have received the profits.

Recommendations

Consider collecting the losses before refunding any collateral to ensure sufficient funds are available to cover total losses. However, the remediation should align with Finding [3.1](#) regarding the separation of users' position collateral from their order collateral.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [224d5627 ↗](#).

3.3. Incorrect initialization of funding accrued for new positions

Target	Orderbook		
Category	Coding Mistakes	Severity	Critical
Likelihood	High	Impact	Critical

Description

At the start of `_fill`, `_accrueFunding` returns early when the user's position size is zero, leaving `p.fundingAccrued` unchanged. Then, `_applyTrade` opens a new position without initializing `p.fundingAccrued` to the current `cumulativeFundingPerUnit`. As a result, on the next funding accrual, the funding due is computed as $(\text{cumulativeFundingPerUnit} - 0) * \text{size}$, charging the user for funding that accrued before the position existed.

The root cause of this issue is failure to initialize `fundingAccrued` for new positions when opening from zero size.

```
function _fill(Order storage maker, Order storage taker, uint128 qty,
    uint128 price) internal {
    _accrueFunding(maker.trader);
    _accrueFunding(taker.trader);
    [...]
}
[...]
function _accrueFunding(address user) internal {
    _accrueGlobalFunding();
    Position storage p = positions[user];
    if (p.size == 0) return;
    int256 due = ((cumulativeFundingPerUnit - p.fundingAccrued) * p.size)
    / 1e18; // sign aware

    // funding is paid to realizedPnl
    p.realizedPnl -= due; // longs pay when delta>0
    p.fundingAccrued = cumulativeFundingPerUnit;
}
```

Impact

Users who open a new position immediately after having zero size will be charged accumulated funding fees since contract inception, potentially causing significant unexpected losses.

Recommendations

Users should only be charged funding during the time they hold a position. Consider updating `p.fundingAccrued` even when the user's position size is zero to prevent charging excessive funding.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [9a831797](#).

3.4. Liquidation denial of service via uint8 overflow in OrderBook::getOpenOrders

Target	OrderBook		
Category	Business Logic	Severity	Critical
Likelihood	High	Impact	Critical

Description

The OrderBook::getOpenOrders function contains a critical integer-overflow vulnerability that can be exploited to prevent liquidations. The function uses a uint8 variable to count open orders, which can only hold values from 0 to 255. When a user accumulates more than 255 open orders, attempting to increment the counter beyond 255 triggers Solidity's automatic arithmetic overflow protection, causing the transaction to revert.

This is the vulnerable code.

```
function getOpenOrders(address user) public view returns (Order[] memory) {
    uint256[] memory allOrders = userOrders[user];
    uint8 count = 0;

    for (uint256 i = 0; i < allOrders.length; ++i) {
        uint256 orderId = allOrders[i];
        Order memory order = orders[orderId];
        if (order.active) {
            count++;
        }
    }
    [...]
}
```

Impact

This vulnerability enables a liquidation denial-of-service (DOS) attack. An attacker can create 256 or more limit orders at extreme prices (e.g., \$1M for ETH) that will never fill, preventing their position from being liquidated even when underwater. The cost is minimal (only gas fees), while the benefit is maintaining leveraged positions without liquidation risk.

The impact is amplified in PerpManager::liquidatePositions, where liquidations are processed in batches. A single exploiting position causes the entire batch to revert, potentially blocking

liquidation of all unhealthy positions in the protocol. As bad debt accumulates across multiple unliquidatable positions, the protocol faces insolvency risk.

Recommendations

Implement a maximum limit on open orders per user to prevent the overflow issue, as simply changing `uint8` to `uint256` would only allow attackers to create thousands of orders causing out-of-gas errors during liquidation. Alternatively, enforce a minimum collateral requirement per open order to make the attack economically unfeasible by creating a significant economic barrier that outweighs the benefit of avoiding liquidation. Consider combining both approaches for layered protection.

Additionally, implement try-catch in `PerpManager::liquidatePositions` to prevent a single failing position from reverting the entire liquidation batch.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [224d5627](#).

3.5. AMM denial of service when proxy orders match as takers

Target	Orderbook		
Category	Business Logic	Severity	Critical
Likelihood	High	Impact	Critical

Description

The `Orderbook::_fill` function incorrectly assumes proxy orders are always makers, hardcoding `false` for the taker's `isProxy` parameter:

```
function _fill(Order storage maker, Order storage taker, uint128 qty,
    uint128 price) internal {
    [...]
    _applyTrade(maker.trader, -quant, price, maker.isProxy);
    _applyTrade(taker.trader, quant, price, false);
}
```

This assumption breaks when AMM bin orders match as takers. If opposing orders already exist in the orderbook, AMM bins will match against them as takers. The positions are then created with `isAMM=false`, completely breaking the AMM's liquidity tracking and management system.

An attacker can exploit this by front-running AMM bin creation, placing orders at exact bin prices to force AMM orders to match as takers. Once bins have positions with `isAMM=false`, they enter a permanently broken state.

Impact

AMM bins with `isAMM=false` are treated as regular user positions, causing the protocol to call vault operations (`vault.unlock`, `vault.moveLocked`) instead of AMM hooks. Since bins do not lock collateral in the vault like regular users, these operations revert due to insufficient funds.

```
function _closePosition(address user) internal {
    [...]
    if (p.isAMM) {
        amm.onClosePosition(user, closingPnl, collateralToRelease);
    } else {
        [...]
        vault.unlock(user, collateralToRelease);
    }
}
```

```

    }
    [...]
}

```

Corrupted bins are immediately marked as liquidatable because `isLiquidatable` reads `vault.lockedBalance` (which is 0 for bins) instead of the bin's actual `lockedCollateral`, making equity evaluate to 0. However, `liquidate` fails when attempting to call `vault.moveLocked` on bins that have no vault collateral:

```

function isLiquidatable(address user) public view returns (bool) {
    [...]

    uint256 lockedCollateral = p.isAMM ? p.lockedCollateral : vault.lockedBalance(user);
    int256 equity = int256(lockedCollateral) + p.realizedPnl - fundingDue + uPnL;
    [...]
    uint256 maint = _maintenanceMarginReq(_abs(p.size), entryPrice);
    return equity < int256(maint);
}
[...]
function liquidate(address user, address caller) internal {
    Position storage p = positions[user];
    [...]
    if (!p.isAMM) {
        vault.moveLocked(user, caller, pen);
        vault.moveLocked(user, address(treasury), collateral - pen);
    } else {
        amm.onLiquidate(user, caller, pen, collateral - pen);
    }
    [...]
}

```

This results in complete AMM DOS as corrupted bin positions cannot be closed, liquidated, shifted, or removed. The entire AMM becomes permanently unusable, blocking all liquidity operations.

Recommendations

Remove the hardcoded false assumption and pass the taker's actual `isProxy` status to `_applyTrade`.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [6f58c7ab](#).

3.6. Mark price does not account for canceled orders

Target	OrderBook		
Category	Business Logic	Severity	Critical
Likelihood	High	Impact	Critical

Description

The function `_getImpactPriceOfBookSide` is invoked by the function `getMarkPrice` when getting the mark price. This function goes through the orderbook, iterating the RB tree and getting the queue of orders at each new price. However, when iterating the orders in the queue, it does not account for canceled orders.

```
function _getImpactPriceOfBookSide(
    bool isLong
) internal view returns (uint256 averagePrice, uint256 cumulativeNotional) {
    ...
    DoubleEndedQueue.Bytes32Deque storage ordersAtPrice = _queue(isLong,
    bestPrice);
    for (uint16 i = 0; i < ordersAtPrice.length(); i++) {
        Order memory order = orders[uint256(ordersAtPrice.at(i))];
        uint256 amount = order.amount;
        uint256 price = order.price;
        uint256 required = NOTIONAL_FOR_SWEEP - cumulativeNotional;
        uint256 orderValueInUsd = _roundToNearestTick((amount * price) / ONE,
        ONE_CENT);
        isFull = orderValueInUsd > required;
        if (isFull) orderValueInUsd = required;
        cumulativeValue += (orderValueInUsd * price);
        cumulativeNotional += orderValueInUsd;
        if (isFull) return (cumulativeValue / NOTIONAL_FOR_SWEEP,
        cumulativeNotional);
    }
    ...
}
```

Impact

Users may craft malicious orders that may impact the mark price without locking any collateral. Allowing for manipulation of the AMM and many orderbook functions.

Recommendations

Add logic to account for canceled orders.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [0dea74e1](#).

3.7. Inconsistency in funding-accrual calculation for isLiquidatable

Target	Orderbook		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

The `_accrueFunding` function updates global funding rates and then accrues funding for a specific user. Meanwhile, the `isLiquidatable` function is a view function that simulates funding accrual before checking if a position has reached the liquidation threshold. These functions contain inconsistencies in their division operation order, as shown in the code snippet.

This causes the `isLiquidatable` function to accrue less funding than what actually occurs.

```
function _accrueGlobalFunding() internal {
    if (block.timestamp <= lastFundingTime) return;
    uint256 dt = block.timestamp - lastFundingTime;
    // funding delta = markPrice * rate * dt / 1e6 (ppm)
    uint256 markPrice = getMarkPrice();
    int256 delta = (int256(markPrice) * int256(fundingRatePpm) * int256(dt))
    / 1e6;
    cumulativeFundingPerUnit += delta; // scale to per base unit // Step 1
    lastFundingTime = uint64(block.timestamp);
    emit FundingUpdated(cumulativeFundingPerUnit);
}
[...]
function _accrueFunding(address user) internal {
    _accrueGlobalFunding();
    Position storage p = positions[user];
    if (p.size == 0) {
        p.fundingAccrued = cumulativeFundingPerUnit;
        return;
    }
    int256 due = ((cumulativeFundingPerUnit - p.fundingAccrued) * p.size) /
    1e18; // sign aware // Step 2

    // funding is paid to realizedPnl
    p.realizedPnl -= due; // longs pay when delta>0
    p.fundingAccrued = cumulativeFundingPerUnit;
}
```

```
[...]
function isLiquidatable(address user) public view returns (bool) {
    // 1. Compute new global funding index
    uint256 dt = block.timestamp - lastFundingTime;
    uint256 markPrice = _getOraclePrice();

    // delta = markPrice * ratePpm * dt / 1e6
    int256 delta = (int256(markPrice) * fundingRatePpm * int256(dt)) / 1e6;

    int256 newCum = cumulativeFundingPerUnit + (delta / int256(ONE)); // Step 1

    // 2. Compute user's funding due
    Position memory p = positions[user];
    int256 fundingDue = (newCum - p.fundingAccrued) * p.size; // sign-aware //
    Step 2
    [...]
}
```

Impact

The inaccurate funding-accrual calculation in the `isLiquidatable` function may inappropriately prevent users from being liquidated when their positions should actually be liquidated.

Recommendations

Consider updating the `isLiquidatable` function to simulate funding accrual consistently with how the `_accrueFunding` function behaves.

```
function isLiquidatable(address user) public view returns (bool) {
    // 1. Compute new global funding index
    uint256 dt = block.timestamp - lastFundingTime;
    uint256 markPrice = _getOraclePrice();

    // delta = markPrice * ratePpm * dt / 1e6
    int256 delta = (int256(markPrice) * fundingRatePpm * int256(dt)) / 1e6;
    int256 newCum = cumulativeFundingPerUnit + (delta / int256(ONE));
    int256 newCum = cumulativeFundingPerUnit + delta;

    // 2. Compute user's funding due
    Position memory p = positions[user];
    int256 fundingDue = (newCum - p.fundingAccrued) * p.size; // sign-aware
    int256 fundingDue = ((newCum - p.fundingAccrued) * p.size) / 1e18; //
```

```
    sign-aware  
    [...]  
}
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [9a831797](#).

3.8. Inconsistent funding accrual across liquidateExt and placeOrder functions

Target	Orderbook		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

The `_accrueFunding` function calls the `_accrueGlobalFunding` function to calculate funding based on the mark price, while the `isLiquidatable` function simulates funding accrual using the oracle price.

```
function _accrueFunding(address user) internal {
    _accrueGlobalFunding();
    Position storage p = positions[user];
    if (p.size == 0) return;
    int256 due = ((cumulativeFundingPerUnit - p.fundingAccrued) * p.size)
    / 1e18; // sign aware

    // funding is paid to realizedPnl
    p.realizedPnl -= due; // longs pay when delta>0
    p.fundingAccrued = cumulativeFundingPerUnit;
}
[...]
function _accrueGlobalFunding() internal {
    if (block.timestamp <= lastFundingTime) return;
    uint256 dt = block.timestamp - lastFundingTime;
    // funding delta = markPrice * rate * dt / 1e6 (ppm)
    uint256 markPrice = getMarkPrice();
    int256 delta = (int256(markPrice) * int256(fundingRatePpm) * int256(dt))
    / 1e6;
    cumulativeFundingPerUnit += delta; // scale to per base unit
    lastFundingTime = uint64(block.timestamp);
    emit FundingUpdated(cumulativeFundingPerUnit);
}
[...]
function isLiquidatable(address user) public view returns (bool) {
    // 1. Compute new global funding index
    uint256 dt = block.timestamp - lastFundingTime;
    uint256 markPrice = _getOraclePrice();
```

```
// delta = markPrice * ratePpm * dt / 1e6
int256 delta = (int256(markPrice) * fundingRatePpm * int256(dt)) / 1e6;
int256 newCum = cumulativeFundingPerUnit + (delta / int256(ONE));

// 2. Compute user's funding due
Position memory p = positions[user];
int256 fundingDue = (newCum - p.fundingAccrued) * p.size; // sign-aware

// 3. Recompute equity including funding

int256 uPnL = _unrealizedPnl(p, markPrice);
uint256 lockedCollateral = p.isAMM ? p.lockedCollateral :
vault.lockedBalance(user);
int256 equity = int256(lockedCollateral) + p.realizedPnl - fundingDue
+ uPnL;
[...]
}
```

The inconsistency occurs because the `placeOrder` function calls `_accrueFunding(user)` before checking with `isLiquidatable(user)`, which relies on mark price. Meanwhile, the `liquidateExt` function directly calls the `liquidate` function, which checks `isLiquidatable(user)` without first calling `_accrueFunding(user)`, relying instead on oracle price.

```
function placeOrder(
    bool isLong,
    uint128 price,
    uint128 amount,
    uint128 maxSlippage, // max slippage in price units
    uint16 leverage, // leverage multiplier (1-100x)
    address user,
    bytes calldata oracleData
) external nonReentrant onlyManager returns (uint256 id) {
    if (price == 0) revert InvalidPrice(price);
    if (price % tickSize != 0) revert InvalidTickSize(price);
    if (amount == 0) revert InvalidAmount(amount);
    if (leverage < minLeverage || leverage > maxLeverage)
        revert InvalidLeverage(leverage);
    bool isProxy = msg.sender == address(amm);
    oracle.updatePriceFeeds(UNDERLYING, STRIKE, oracleData);

    _accrueFunding(user);

    // external bots will handle liquidation but this is still useful as a
    // safety check

    // we don't return early as we should still allow them to place an order if
```

```
    they have collateral
    // required after this. If they don't it will revert as expected
    if (isLiquidatable(user)) liquidate(user, user);
    [...]
}
[...]
function liquidateExt(address user, address caller) external onlyManager {
    liquidate(user, caller);
}
[...]
function liquidate(address user, address caller) internal {
    Position storage p = positions[user];

    int256 size = p.size;
    if (size == 0) revert NoPosition();
    if (!isLiquidatable(user)) revert NotLiquidatable();
    [...]
}
```

Impact

Users might avoid liquidation through the `liquidateExt` call but get force-liquidated when using `placeOrder`, or vice versa, depending on differences between mark price and oracle price.

Recommendations

Revise the funding accrual and liquidation mechanisms in the `liquidateExt` and `placeOrder` functions to ensure they handle these processes consistently.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [9a831797](#).

3.9. Potential liquidation when matching same user orders

Target	Orderbook		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

Users who place opposite order and match them with their own, while having an existing nonzero position, will be immediately liquidatable.

The root cause of this issue lies in the `_applyTrade` function on the taker side. The `closing` variable is multiplied with `_sign(qty)`, which flips the closing size. This causes `p.entryNotional` to increase instead of decrease, resulting in the position's `entryNotional` being updated to an excessively high value.

```
function _fill(Order storage maker, Order storage taker, uint128 qty,
    uint128 price) internal {
    [...]

    // update positions
    _applyTrade(maker.trader, -quant, price, maker.isProxy);
    _applyTrade(taker.trader, quant, price, false);

    emit OrderMatched(maker.id, taker.id, qty, price);
}
[...]
function _applyTrade(address trader, int256 qty, uint128 price, bool isProxy)
    internal {
    Position storage p = positions[trader];
    // If order is long, taker increases long -> qty positive; maker side we
    sent negative
    if (isProxy) p.isAMM = true;
    int256 oldSize = p.size;
    int256 newSize = oldSize + qty;

    bool sameSide = (oldSize >= 0 && qty >= 0) || (oldSize <= 0 && qty <= 0) ||
    oldSize == 0;

    // if same direction -> simple VWAP add
    if (sameSide) {
        // same sign or zero
```



```

        p.entryNotional += int256((uint256(price) * uint256(_abs(qty))) / ONE)
        * _sign(qty);
        p.size = newSize;
    } else {
        int256 closing = _minAbs(oldSize, qty) * _sign(qty); // qty sign
                    indicates the new direction

        int256 avgEntry = oldSize == 0 ? int256(uint256(price)) :
        (p.entryNotional * 1e18) / oldSize;
        int256 realized = (((int256(uint256(price)) - avgEntry) * closing)
        / 1e18) * _sign(oldSize);
        p.realizedPnl += realized;
        // Reduce entryNotional proportional
        p.entryNotional -= ((avgEntry * closing) / 1e18) * _sign(oldSize);
        p.size = newSize;
    }

    // if the position size gets set to zero (for example matched order causes
    breakeven)
    // we close and run pnl calcs plus release any collateral or pay any fees
    etc.
    bool shouldClose = oldSize != 0 && newSize == 0;
    if (shouldClose) _closePosition(trader);
}

```

This leads to liquidation because the maintenance margin requirement becomes too high.

```

function isLiquidatable(address user) public view returns (bool) {
    [...]

    // 4. Compare vs maintenance - use entry price for maintenance calculation
    // This ensures that as price drops, maintenance requirement doesn't drop
    proportionally
    uint256 entryPrice = 0;
    if (p.size != 0) {
        // Calculate absolute entry price from entryNotional and size
        entryPrice = uint256(_abs((p.entryNotional * 1e18) / p.size));
    } else {
        entryPrice = markPrice; // Fallback if no position
    }
    uint256 maint = _maintenanceMarginReq(_abs(p.size), entryPrice);
    return equity < int256(maint);
}

```

Impact

When matched orders from the same user are processed, they can potentially make the position immediately liquidatable, depending on how `entryNotional` is updated.

Recommendations

Consider using the closing size directly without applying the sign of `qty` to ensure that the position's `entryNotional` is updated accurately.

```
function _applyTrade(address trader, int256 qty, uint128 price, bool isProxy)
internal {
    [...]
    // if same direction -> simple VWAP add
    if (sameSide) {
        // same sign or zero
        p.entryNotional += int256((uint256(price) * uint256(_abs(qty))) / ONE)
        * _sign(qty);
        p.size = newSize;
    } else {
        int256 closing = _minAbs(oldSize, qty) * _sign(qty); // qty sign
        // indicates the new direction
        int256 closing = _minAbs(oldSize, qty)

        int256 avgEntry = oldSize == 0 ? int256(uint256(price)) :
        (p.entryNotional * 1e18) / oldSize;
        int256 realized = (((int256(uint256(price)) - avgEntry) * closing)
        / 1e18) * _sign(oldSize);
        p.realizedPnl += realized;
        // Reduce entryNotional proportional
        p.entryNotional -= ((avgEntry * closing) / 1e18) * _sign(oldSize);
        p.size = newSize;
    }
    [...]
}
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [c8067ed9](#).

3.10. Inaccurate AMM liquidity accounting

Target	AMM		
Category	Business Logic	Severity	High
Likelihood	Medium	Impact	High

Description

The AMM contract allows the bin manager to create bins that generate profits or losses, which are tracked using `poolPnl`. However, the `withdrawFromVault` and `depositToVault` functions, which transfer liquidity between the AMM and the vault, consider only the liquidity without accounting for `poolPnl`. This prevents an accurate reflection of the actual available liquidity.

```
function withdrawFromVault(uint256 _amount) external onlyOwner {
    genesisVault.withdraw(_amount);
    liquidity += _amount;
}

function depositToVault(uint256 _amount) external onlyOwner {
    genesisVault.deposit(_amount);
    liquidity -= _amount;
}

[...]

function onClosePosition(address binAddress, int256 profit, uint256 amount)
    external {
    if (msg.sender != address(orderbook)) revert NotOrderbook(msg.sender);
    uint128 price = binPrices[binAddress];
    BinDetails storage bin = bins[price];
    bin.lockedLiquidity -= amount;
    poolPnl += profit;
    if (profit > 0) {
        bin.liquidity += uint256(profit);
        treasury.withdraw(address(usdc), address(this), uint256(profit));
    } else {
        bin.liquidity -= uint256(-profit);
        usdc.transfer(address(treasury), uint256(-profit));
    }
}

function onLiquidate(address binAddress, address caller, uint256 penalty,
    uint256 loss) external {
    if (msg.sender != address(orderbook)) revert NotOrderbook(msg.sender);
```

```
uint128 price = binPrices[binAddress];
BinDetails storage bin = bins[price];
uint256 totalLoss = penalty + loss;
bin.lockedLiquidity -= totalLoss;
bin.liquidity -= totalLoss;
poolPnl -= int256(totalLoss);
usdc.transfer(address(caller), penalty);
usdc.transfer(address(treasury), loss);
}
```

Impact

This creates a discrepancy between actual available funds and tracked liquidity. As a result, the owner cannot move the actual available funds from the AMM contract to the vault, even after bins are completely removed.

Recommendations

Consider reimplementing the liquidity tracking system to ensure owners can correctly move liquidity between the AMM and vault. The liquidity should properly update to reflect profits and losses experienced by the AMM.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [e5cae625](#).

3.11. Insufficient test-suite code coverage

Target	Multiple Contracts		
Category	Code Maturity	Severity	High
Likelihood	High	Impact	High

Description

When building a complex contract ecosystem with multiple moving parts and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios. Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

The test coverage for this project should be expanded to include all contracts, not just surface-level functions. It is important to test critical invariants to ensure security and confirm that the system behaves as intended.

Therefore, we recommend building a rigorous test suite that includes all contracts to ensure that the system operates securely and as intended.

Good test coverage has multiple effects.

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in your product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point seems contradictory, given the time investment to create and maintain tests. To expand upon that, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks something if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence. Tests have your back here. They are an indicator that the existing functionality *most likely* was not broken by your change to the code.

Impact

Because correctness is so critically important when developing smart contracts, we recommend that all projects strive for 100% code coverage.

Testing should be an essential part of the software development life cycle. No matter how simple a function may be, untested code is always prone to bugs.

Recommendations

Although the test coverage of some parts of the system is quite good and includes numerous cases, the overall test coverage is still insufficient. As a result, several findings in this audit would have been discovered with simple test cases.

Therefore, we strongly recommend to implement test cases for all the contracts and functions.

Specific functions and areas that can be improved include the following:

- Several areas within Orderbook, including
 - AMM as both maker and taker order
 - funding-fees accrual (both positive and negative)
 - mark-price behavior in low- and high-liquidity conditions
 - mark-price manipulation
 - liquidation during order placement
- Several areas within AMM, including
 - position liquidation
 - liquidity movement between the AMM and vault
 - removal of the bin when an order is partially filled
- Bin management with various configurations (in BinManager)
- Cross-contract interactions
- End-to-end trading workflows (including AMM and non-AMM)

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [01b558d8](#) that cover the missing cases identified in the threat model. Since our report represents a point-in-time snapshot of the codebase, the Threat Model reflects an earlier commit—before these additional tests were added.

3.12. Insufficient treasury funds could prevent closing profitable positions

Target	Orderbook		
Category	Business Logic	Severity	High
Likelihood	Medium	Impact	High

Description

The Orderbook contract's position-closure mechanism requires the Treasury to have sufficient funds to pay out profits to winning users. When a user closes a position with positive PnL, the `_closePosition` function attempts to withdraw funds from the Treasury and credit them to the user's vault balance. However, the protocol lacks a mechanism to ensure the Treasury maintains sufficient reserves to cover potential profits across all open positions. While losses are limited by liquidation and flow into the Treasury, profits can grow unbounded (particularly with high leverage) and must be paid from the Treasury's limited reserves.

This creates a structural imbalance where the Treasury can become insolvent when multiple positions are profitable simultaneously or when large leveraged positions generate outsized gains.

Impact

Users with winning positions cannot close their positions and realize profits when the Treasury has insufficient funds. This effectively traps user collateral and unrealized gains in the protocol.

Recommendations

We cannot provide a complete solution without breaking the contract's functionality. However, consider adding a mechanism to ensure the Treasury maintains sufficient funds, which would guarantee that profitable positions can be closed at any time.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [afcd2759](#).

3.13. No position-size validation allows unlimited collateral-free order creation

Target	Orderbook		
Category	Coding Mistakes	Severity	High
Likelihood	High	Impact	High

Description

Both `Orderbook::placeOrder` and `PerpManager::openPosition` do not validate minimum position sizes, allowing attackers to create orders with arbitrarily small amounts. When `Orderbook::placeOrder` calculates the collateral required, it calls `Orderbook::_marginReq`, which for small amounts, rounds down to zero, resulting in zero collateral requirement.

```
function placeOrder(
    bool isLong,
    uint128 price,
    uint128 amount,
    uint128 maxSlippage, // max slippage in price units
    uint16 leverage, // leverage multiplier (1-100x)
    address user,
    bytes calldata oracleData
) external nonReentrant onlyManager returns (uint256 id) {
    [...]
    uint256 collateralRequired = _marginReq(amount, uint256(price), leverage);
    [...]
}
[...]
function _marginReq(uint256 absSize, uint256 price, uint16 leverage)
    internal pure returns (uint256) {
    return ((absSize * price) / leverage) / ONE;
}
```

Attackers can exploit this by creating dust orders with amounts small enough that the collateral calculation rounds to zero, bypassing all margin requirements at no economic cost.

Impact

Attackers can place unlimited collateral-free dust orders at no economic cost. This enables attackers to spam dust orders at matching prices, causing the matching engine to waste all its 16 iterations (`MATCH_LOOP_CAP`) processing worthless dust orders instead of legitimate liquidity,

preventing orders from being completely filled. In addition, the zero-collateral nature also amplifies other attack vectors and can lead to loss of funds through blocked liquidations or unfilled orders.

Recommendations

Enforce minimum order size to prevent rounding to zero collateral and minimum collateral requirements in `Orderbook::placeOrder` to make spam attacks economically impractical:

```
function placeOrder(
    bool isLong,
    uint128 price,
    uint128 amount,
    uint128 maxSlippage,
    uint16 leverage,
    address user,
    bytes calldata oracleData
) external nonReentrant onlyManager returns (uint256 id) {
    if (price == 0) revert InvalidPrice(price);
    if (price % tickSize != 0) revert InvalidTickSize(price);
    if (amount == 0) revert InvalidAmount(amount);
    if (amount < MIN_ORDER_SIZE) revert AmountTooSmall(amount);
    if (leverage < minLeverage || leverage > maxLeverage)
        revert InvalidLeverage(leverage);
    [...]
    uint256 collateralRequired = _marginReq(amount, uint256(price), leverage);
    if (collateralRequired < MIN_COLLATERAL) revert InsufficientCollateral(
        collateralRequired);
    [...]
}
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [11ee20e9](#).

3.14. Order cancellation can cause denial of service in matching

Target	OrderBook		
Category	Business Logic	Severity	High
Likelihood	High	Impact	High

Description

In the function `_matchIncoming`, responsible for handling orderbook matching, there is an edge case in which canceling orders can result in a corrupted queue state.

First, two identically priced orders can be added to the orderbook, such that they are in the same queue. Subsequently, the order at the front of the queue is canceled; this is done first to avoid it being removed from the queue. Then, the order at the back of the queue is canceled, such that it is removed from the queue.

```
function cancelOrder(uint256 id, address user) public onlyManager {
    QueueWithVolume storage queueWithVolume = _queueWithVolume(order.isLong,
        order.price);
    ...
    bool queueHasLength = q.length() > 0;
    bool isTailEndOfQueue = queueHasLength && _isOrderAtTailOfQueue(q,
        uint256(order.id));
    if (isTailEndOfQueue) {
        q.popBack();
        if (q.length() == 0) _removePriceFromTree(order.isLong, order.price);
    }
    emit OrderCancelled(id, remaining);
}
```

Finally, the queue is in such a state that it contains one order that is canceled but not yet removed. It is the job of `_matchIncoming` to remove this order when it finds it. This is executed by a `popFront`. However, even though the queue will now be empty, the entry in the RB tree still persists.

```
function _matchIncoming(uint256 takerId) internal {
    ...
    DoubleEndedQueue.Bytes32Deque storage ordersAtPrice = _queue(!isLong,
        bestPrice); // grab the opposite queue

    uint256 makerId = uint256(ordersAtPrice.front());
    Order storage makerOrder = orders[makerId];
```

```
if (!_isFillable(makerOrder)) {  
    ordersAtPrice.popFront();  
    continue;  
}  
...  
}
```

When the matching process continues, it will iterate the RB tree, finding the current price entry, and when it tries to iterate the queue using `ordersAtPrice.front()`, it will revert as the queue is empty.

Impact

This corruption of the queue could happen randomly as users interact with the orderbook, after which it would be frozen as with every attempt to match an order, users would revert. This would result in a denial of service.

Recommendations

Remove the RB tree entry if the queue is empty after being popped.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [05f4cadf](#).

3.15. Inflexible price-staleness check for diverse asset types

Target	PythFeeds, StorkFeeds		
Category	Integration Risks	Severity	High
Likelihood	Medium	Impact	High

Description

The PythFeeds and StorkFeeds oracle contracts implement a single, universal `maxOracleDelay` parameter that applies to all asset pairs, despite different assets having vastly different price-update frequencies and volatility characteristics.

Impact

Attackers can monitor multiple asset pairs simultaneously, exploiting whichever assets currently have the widest gap between stale oracle prices and real market prices, maximizing extraction across the entire protocol.

Recommendations

Consider assigning a specific `maxOracleDelay` per asset rather than using a single value. This would allow the system to fine-tune staleness tolerances based on the characteristics of each individual asset. For example, high-volatility tokens could have tighter constraints, while stablecoins or slower-moving assets could be permitted longer validity periods.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [8c18bf6f](#).

3.16. Risk of repeatedly creating initial bins exceed the configuration limit

Target	BinManager		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `createInitialBins` function allows the manager to create bins on the same side only once, according to the `binConfig.totalBins` limit. It also permits creating bins again when previous bins are matched or completely removed. However, the `createInitialBins` function fails to check which bins are still active.

Impact

This allows the manager to mistakenly create bins exceeding the `totalBins` configuration limit. This risk is particularly significant when the initial bins are created on opposite sides, as they could potentially match with each other — a situation explicitly restricted in the `_calculateNewBinPrices` function.

```
function _calculateNewBinPrices(uint128 currentPrice, bool isLong)
    internal view returns (uint128[] memory) {
        uint256 totalBins = binConfig.totalBins;
        uint128[] memory newPrices = new uint128[](totalBins);

        uint256 spreadPerBin = (currentPrice * binConfig.binSpread) / 10000;

        // We need to create bins on one side else they get matched with each other
        for (uint256 i = 0; i < binConfig.totalBins; i++) {
            if (isLong) {
                uint128 binPrice = currentPrice + uint128(spreadPerBin * (i + 1));
                newPrices[i] = _roundToTickSize(binPrice);
            } else {
                uint128 binPrice = currentPrice - uint128(spreadPerBin * (i + 1));
                newPrices[i] = _roundToTickSize(binPrice);
            }
        }
        return newPrices;
    }
```

Recommendations

Add validation to revert when the bins are still active, ensuring bins are completely removed before initializing the new set of bins.

```
function createInitialBins(  
    uint256[] calldata liquidityAmounts,  
    bool isLong,  
    bytes calldata oracleData  
) external onlyManager {  
    if (amm.getActiveBinPrices().length > 0) {  
        revert BinsAlreadyCreated();  
    }  
    [...]  
}
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [e5cae625](#).

3.17. Unsafe casting of negative rates causes underflow in OracleManager::getUnderlyingPrice

Target	OracleManager		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `OracleManager::getUnderlyingPrice` function casts the normalized price to `uint256` without checking if it is negative. For assets with `isNegativeRateSupported` enabled like funding rates (e.g., `HL_BTC_CURRENT_FUNDING`), `_normalizePrice` can return a negative value when the magnitude of the negative price exceeds the offset (i.e., `offset + price < 0`):

```
function getUnderlyingPrice(bytes32 _underlying, bytes32 _strike)
    public view override returns (uint256) {
    [...]
    int8 decimal = 0;
    int256 price = 0;
    (price, decimal) = aggregator.getPrice(_underlying, _strike);
    int256 offset;
    if (isNegativeRateSupported[_underlying][_strike]) {
        if (decimal < 0) {
            // offset is 10 ** decimal
            offset = int256(10 ** uint256(int256(-decimal)));
        } else {
            offset = int256(10 ** uint256(int256(decimal)));
        }
        price = _normalizePrice(offset + price, decimal);
    } else {
        price = _normalizePrice(price, decimal);
    }

    return uint256(price);
}
```

While unlikely under normal market conditions, extreme market fluctuations or certain asset types with negative rates could lead to this scenario. When a negative `int256` value is cast to `uint256`, the two's complement representation is reinterpreted as a very large positive number.

Impact

The incorrectly cast price causes catastrophic miscalculations across all protocol operations relying on price data, including position valuations, liquidation thresholds, collateral requirements, and PnL calculations. This leads to failed or premature liquidations, incorrect margin requirements that allow overleveraged positions, and direct loss of funds for users.

Recommendations

Add validation in `getUnderlyingPrice` to revert when the normalized price is negative before casting to `uint256`.

```
function getUnderlyingPrice(bytes32 _underlying, bytes32 _strike)
    public view override returns (uint256) {
    [...]
    int8 decimal = 0;
    int256 price = 0;
    (price, decimal) = aggregator.getPrice(_underlying, _strike);
    int256 offset;
    if (isNegativeRateSupported[_underlying][_strike]) {
        if (decimal < 0) {
            // offset is 10 ** decimal
            offset = int256(10 ** uint256(int256(-decimal)));
        } else {
            offset = int256(10 ** uint256(int256(decimal)));
        }
        price = _normalizePrice(offset + price, decimal);
    } else {
        price = _normalizePrice(price, decimal);
    }

    if (price < 0) revert NegativePrice(price);
    return uint256(price);
}
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [1c0b72fba](#).

3.18. Oracle staleness limit allows extended price delays

Target	PythFeeds, StorkFeeds		
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

The `changeMaxOracleDelay` function validates against a hardcoded limit of 60,000 seconds (16.67 hours), enabling administrators to configure dangerously high price-staleness thresholds that enable the protocol to accept outdated price data:

```
function setOracleDelay(uint256 _oracleDelay) public onlyOwner(msg.sender) {
    require(_oracleDelay <= maxOracleDelay, "Invalid oracle delay");
    oracleDelay = _oracleDelay;
}

/**
 * @notice Update the maximum allowed oracle delay limit
 * @param _maxOracleDelay New maximum delay limit in seconds
 * @dev Must be less than 120 seconds, affects validation for setOracleDelay
 *      calls
 */
function changeMaxOracleDelay(uint256 _maxOracleDelay)
    public onlyOwner(msg.sender) {
    require(_maxOracleDelay < 60000, "Invalid max oracle delay"); // we should
    be able to customise this
    maxOracleDelay = _maxOracleDelay;
}
```

Impact

Accepting prices up to 16 hours old causes incorrect position valuations and liquidation decisions. Attackers can exploit the price discrepancy between stale oracle data and actual market prices for risk-free arbitrage, while unreliable collateral requirements and PnL calculations can lead to protocol insolvency.

Recommendations

Correct the validation limit in `PythFeeds::changeMaxOracleDelay` and `StorkFeeds::changeMaxOracleDelay` to match the documented requirement of 120 seconds. Consider using a constant to store the `MAX_ORACLE_DELAY_CAP`. For production deployments, a 60–120 second maximum ensures prices remain reasonably fresh.

```
function changeMaxOracleDelay(uint256 _maxOracleDelay)
    public onlyOwner(msg.sender) {
        require(_maxOracleDelay < 60000, "Invalid max oracle delay");
        require(_maxOracleDelay < MAX_ORACLE_DELAY_CAP, "Invalid max oracle delay"
        );
        maxOracleDelay = _maxOracleDelay;
    }
```

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [0f75de6e](#).

3.19. Griefing attack via updatePriceFeedsIfNecessary function

Target	PythFeeds, StorkFeeds		
Category	Business Logic	Severity	Low
Likelihood	High	Impact	Low

Description

The `PythFeeds::updatePriceFeedsIfNecessary` and `StorkFeeds::updatePriceFeedsIfNecessary` functions allow anyone to submit arbitrary oracle-update data as long as it is valid while the contract pays all fees from its balance. The functions lack validation to ensure only protocol-required price feeds are included:

```
function updatePriceFeedsIfNecessary(bytes calldata updateData)
    external override {
        bytes[] memory pythUpdateData = abi.decode(updateData, (bytes[]));
        uint256 fee = pythOracle.getUpdateFee(pythUpdateData);
        if (fee > address(this).balance) revert NotEnoughFundsForFee();
        pythOracle.updatePriceFeeds{value: fee}(pythUpdateData); // No access
        control
    }

function updatePriceFeedsIfNecessary(bytes calldata updateData)
    external override {
        StorkStructs.TemporalNumericValueInput[] memory storkUpdateData
        = abi.decode(
            updateData,
            (StorkStructs.TemporalNumericValueInput[])
        );
        uint fee = stork.getUpdateFeeV1(storkUpdateData);
        if (fee > address(this).balance) revert NotEnoughFundsForFee();
        stork.updateTemporalNumericValuesV1{value: fee}(storkUpdateData); // No
        access control
    }
```

An attacker can craft update data containing multiple unnecessary price feeds (obtained from Pyth/Stork APIs with valid signatures) when only a single underlying/strike pair is needed, allowing sustained griefing through repeated operations to drain the contract's balance.

Impact

While depleting the contract's balance requires sustained attacks over time due to minimal per-update fees, the consequences are severe once the balance is exhausted. All oracle-dependent operations experience complete DOS trading halts, users cannot close positions, and liquidations become impossible. The inability to liquidate underwater positions leads to bad debt accumulation, creating protocol insolvency risk.

Since Pyth/Stork charges different fees across chains, the severity of this issue depends on how much is actually charged.

Recommendations

There are two solutions to mitigate this issue. Consider these options to align with your business requirements.

1. Restrict access to the `updatePriceFeedsIfNecessary` function to only the `OracleManager` contract. This prevents an attacker from directly calling the function for griefing attacks. However, there is no way to verify that the `updateData` contains only prices for a single underlying/strike pair. This means the solution does not completely solve the problem, as an attacker could still craft update data containing multiple unnecessary price feeds and call legitimate functions to drain the contract's balance.
2. Shift oracle fee-payment responsibility to users by making the entire call chain payable. This will completely prevent an attacker from conducting griefing attacks.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [5bf12c64](#).

3.20. PythFeeds ignores confidence intervals, violating best practices

Target	PythFeeds		
Category	Integration Risks	Severity	Low
Likelihood	High	Impact	Low

Description

The `PythFeeds::getPrice` function retrieves price data without validating the confidence interval, violating [Pyth's best practices](#) ⁷. Confidence intervals widen during high-volatility or low-liquidity periods, signaling reduced data reliability:

```
function getPrice(
    bytes32 _underlying,
    bytes32 _strike
) external view override returns (int256 price, int8 decimals) {
    bytes32 id = assetMap[_underlying][_strike];
    if (id == bytes32(0)) {
        revert NoAsset(id);
    }
    PythStructs.Price memory priceDetails = pythOracle.getPriceNoOlderThan(id,
    oracleDelay);
    // missing confidence validation
    price = int256(int64(priceDetails.price));
    decimals = int8(int32(priceDetails.expo));
}
```

The protocol uses this potentially unreliable price data for critical operations including position valuations, liquidations, and PnL calculations without quality verification.

Impact

Using prices with wide confidence intervals during market volatility or low liquidity can result in inaccurate pricing decisions. This may cause positions to be liquidated at suboptimal prices or allow underwater positions to remain open longer than appropriate. While the staleness check provides some protection, the protocol lacks defense against low-quality price data during volatile or illiquid market conditions.

Recommendations

Add confidence-interval validation following Pyth's best practices. Define a maximum acceptable confidence threshold and revert when exceeded.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [8c18bf6f](#).

3.21. Arbitrage and sandwiching

Target	Orderbook		
Category	Integration Risks	Severity	Indeterminate
Likelihood	Indeterminate	Impact	Indeterminate

Description

Many actions done by the AMM/Orderbook rely on the mark price, which is a partial result of orderbook liquidity and oracle data.

There are possibilities for a malicious attacker to

- atomically sandwich a transactions to favorably move/impact the mark price into their desired direction, and
- sandwich AMM operations to potentially grief AMM bin centering/shifting.

Impact

This could impact the validity of calculations in the exchange, the liquidity in the orderbook, the validity of actions taken in the orderbook. Potentially resulting in mispricing or even in the worst case lost funds.

Recommendations

Consider only using Oracle data for low liquidity assets or implementing some sort of Vetting system for the addition of new assets.

Remediation

This issue has been acknowledged by Nunchi, and a fix was implemented in commit [011ca85e](#).

This was implemented by ensuring that the bands for the pricing was generated by the oracle price and not the mark price.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Audit assumptions

We audited this protocol under the assumption that the development team intends to deploy with specific token configurations. Any deviations from these assumptions may introduce vulnerabilities or functional issues that are not covered by this audit.

Collateral token decimals

The protocol is architected with the assumption that collateral tokens use 18 decimals. All contract implementations reflect this design:

- **Core constants:** All calculations use 1e18 as the base unit (ONE = 1e18 in Orderbook and AMM contracts)
- **Price scaling:** Prices are scaled to 18 decimals (uint256 public constant ONE = 1e18; // base unit for size/price maths)
- **Position data:** Position sizes, notional values, and PnL calculations assume 18 decimals precision
- **Margin calculations:** Initial margin, maintenance margin, and liquidation thresholds use 18 decimals math
- **Oracle integration:** The OracleManager normalizes all external price feeds to 18 decimals, expecting collateral to match this precision
- **Vault operations:** Deposit, withdrawal, and balance tracking in Vault.sol assume 18 decimals token operations
- **Test suite** The test suite reinforces this 18 decimals assumption, specifically mocking USDC as an 18 decimal asset

This design choice permeates the entire protocol including Orderbook, OracleManager, Vault, AMM and BinManager contracts.

The development team must ensure deployment uses 18 decimals compliant collateral tokens or implements appropriate decimal handling. If decimal normalization is implemented differently from 18 decimals, all tests should be updated to validate this configuration, and the implementation must undergo an audit before deploying to production.

During the remediation, support was added to handle normalization of non 18-decimal assets ([0950e76b](#)). However, this introduced a potential issue with assets whose prices are not offset and are very low. Specifically within the function `_accrueGlobalFunding`

```
function _accrueGlobalFunding() internal {  
    ...  
    uint256 markPrice = getMarkPrice();
```



```
int256 delta = (int256(markPrice) * int256(fundingRatePpm) * int256(dt))
/ 1e6;
cumulativeFundingPerUnit =
    cumulativeFundingPerUnit +
    int256(_normalise(delta > 0 ? uint256(delta) : uint256(-delta),
BASE_DECIMALS, usdcDecimals)) *
    _sign(delta); // scale to per base unit
    ...
}
```

For frequent enough accruals in popular markets, and a reasonable funding rate, the global accrual can be truncated, accruing at a much lower rate than expected. This can range from negligible to not accruing at all. The team mentioned that global funding will change to a window based system in the near future, where the user's size will be taken into consideration as such preventing this truncation.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

Nunchi added the missing testcases during remediation between commits [1130f715](#) and [01b558d8](#). Because our report is based on a point-in-time snapshot of the code, the tests listed below were not present in our original review.

5.1. Module: Orderbook.sol

Function: `cancelOrder(uint256 id, address user)`

This function is only available to the manager role assigned to PerpManager and AMM contracts. It allows a user to cancel an active order and refund collateral proportional to the unfilled amount.

Inputs

- `id`
 - **Control:** Full control.
 - **Constraints:** N/A.
 - **Impact:** The ID of the order to cancel.
- `user`
 - **Control:** The caller of the `PerpManager.cancelOrder` function or AMM contract.
 - **Constraints:** The order has to be active and belong to the user.
 - **Impact:** The address of the user who is canceling the order.

Branches and code coverage

Intended branches

- Cancel an order successfully when it is partially filled.
 - ☐ Test coverage
- Call the `AMM.onCancelOrder` function when the order belongs to the AMM.
 - ☐ Test coverage
- The volume of the order in the queue should be updated correctly.
 - ☐ Test coverage

- The order should be removed from the queue if it is at the tail of the queue.
 - ☐ Test coverage
- Emit the OrderCancelled event when the order is canceled.
 - ☐ Test coverage

Negative behavior

- Revert when canceling an order when it is fully filled.
 - ☐ Negative test

Function call analysis

- vault.unlock(user, refund)
 - **What is controllable?** user and refund.
 - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the locked balance is less than the refund amount.
- amm.onCancelOrder(order.trader, refund)
 - **What is controllable?** order.trader and refund.
 - **If the return value is controllable, how is it used and how can it go wrong?** There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the caller is not the Orderbook contract.

Function: closePosition(address user, bytes oracleData)

This function is only available to the manager role assigned to PerpManager and AMM contracts. It allows a user to close their position, which means the user can take profit or loss.

Inputs

- user
 - **Control:** The caller of the PerpManager.closePosition function or AMM contract.
 - **Constraints:** The user has to have a position.
 - **Impact:** The address of the user who is closing the position.
- oracleData
 - **Control:** Full control.

- **Constraints:** The oracle data has to be valid, checked by the `IOracle.updatePriceFeedsIfNecessary` call.
- **Impact:** The oracle data used to update the price feeds.

Branches and code coverage

Intended branches

- The funding should be accrued correctly when closing a position.
 - ☐ Test coverage
- Call the `AMM.onClosePosition` function when the position belongs to the AMM.
 - ☐ Test coverage
- Add profit to the user balance in the vault when the position is profitable.
 - ☐ Test coverage
- Move the loss amount from the user to the treasury when the position is a loss.
 - ☐ Test coverage
- Take the loss amount if it is less than the locked collateral when the position is a loss.
 - ☐ Test coverage
- Take the total locked collateral if it is less than the loss amount when the position is a loss.
 - ☐ Test coverage
- Unlock the remaining locked collateral when the position is closed.
 - ☐ Test coverage
- Emit the `Profit` event when the position is profitable.
 - ☐ Test coverage
- Emit the `Loss` event when the position is a loss.
 - ☐ Test coverage
- Emit the `PositionClosed` event when the position is closed.
 - ☐ Test coverage

Negative behavior

- Revert when the treasury does not have sufficient funds to pay the profit.
 - ☐ Negative test

Function call analysis

- `oracle.updatePriceFeeds(UNDERLYING, STRIKE, oracleData)`
 - **What is controllable?** `oracleData`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the `oracleData` is invalid.

Function: `liquidateExt(address user, address caller)`

This function is only available to the manager role assigned to `PerpManager`. It allows a caller to liquidate an unhealthy position and get a penalty reward.

Inputs

- `user`
 - **Control:** Full control.
 - **Constraints:** The user has to have a position, and it has reached the liquidation criteria.
 - **Impact:** The address of the user who is liquidated.
- `caller`
 - **Control:** Full control.
 - **Constraints:** N/A.
 - **Impact:** The address of the liquidator who receives the penalty reward.

Branches and code coverage

Intended branches

- The penalty reward should be calculated correctly based on liquidate penalty BPS.
 - ☐ Test coverage
- Should call the `onLiquidate` function when the position belongs to the AMM.
 - ☐ Test coverage
- Should emit the `Loss` event when the position is liquidated.
 - ☐ Test coverage

Function: `placeOrder(bool isLong, uint128 price, uint128 amount, uint128 maxSlippage, uint16 leverage, address user, bytes oracleData)`

This function is only available to the manager role assigned to the `PerpManager` and `AMM` contracts. It allows a user to place a limit order in the orderbook with leverage.

Inputs

- `isLong`
 - **Control:** Full control.
 - **Constraints:** N/A.
 - **Impact:** The side of the order (long or short).
- `price`
 - **Control:** Full control.
 - **Constraints:** Has to align with tick size.
 - **Impact:** Accepted price of the order.
- `amount`
 - **Control:** Full control.
 - **Constraints:** N/A.
 - **Impact:** The amount of the order.
- `maxSlippage`
 - **Control:** Full control.
 - **Constraints:** N/A.
 - **Impact:** The maximum slippage of the order.
- `leverage`
 - **Control:** Full control.
 - **Constraints:** Has to be between `minLeverage` and `maxLeverage`.
 - **Impact:** The leverage of the order.
- `user`
 - **Control:** The caller of the `PerpManager.openPosition` function or `AMM` contract.
 - **Constraints:** N/A.
 - **Impact:** The address of the user who is opening an order.
- `oracleData`
 - **Control:** Full control.
 - **Constraints:** The oracle data has to be valid, checked by the `IOracle.updatePriceFeedsIfNecessary` call.
 - **Impact:** The oracle data used to update the price feeds.

Branches and code coverage

Intended branches

- The funding should be accrued correctly when placing an order.
 - ☐ Test coverage
- Liquidate when the current position is liquidatable before placing a new order.
 - ☐ Test coverage
- Do not lock collateral when the order is placed by the AMM.
 - ☐ Test coverage
- Orders should be placed and matched successfully when maker and taker are the same user.
 - ☐ Test coverage

Negative behavior

- Revert when placing an order with an invalid price, not matching tickSize.
 - ☐ Negative test
- Revert when placing an order with invalid leverage.
 - ☐ Negative test
- Revert when the feed contract does not have enough funds for the fee when updating price feeds.
 - ☐ Negative test
- Revert when the margin is insufficient.
 - ☐ Negative test
- Revert when the user does not have enough balance to lock collateral.
 - ☐ Negative test

Function call analysis

- `oracle.updatePriceFeeds(UNDERLYING, STRIKE, oracleData)`
 - **What is controllable?** `oracleData`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
There is no return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reverts if the `oracleData` is invalid.

6. Assessment Results

During our assessment on the scoped Yield Perps contracts, we discovered 21 findings. Six critical issues were found. Nine were of high impact, three were of medium impact, and two were of low impact.

We often observe a high number of findings in projects undergoing rapid development. Our recommendation to the Nunchi team is to implement a security-focused development workflow. This includes augmenting the codebase with a comprehensive test suite to ensure proper behavior under real-world conditions.

The project consists of multiple interconnected components including PerpManager, Orderbook, Vault, Treasury, and OracleManager. Additionally, the system can be managed through components like BinManager, which provides liquidity to desired Orderbooks.

These components interact extensively and can significantly influence many edge cases. For example, the AMM contract can provide liquidity to the Orderbook under exclusive conditions that differ from those of regular users. This creates a high level of complexity, with numerous states and contextual interactions that are difficult to fully track mentally during development or review.

However, the existing testing suite used during the assessment is limited in its coverage and does not fully test all components of the smart contracts. Although the existing test coverage is broad, it is still insufficient for such a complex system. For a protocol with this level of interaction, testing becomes especially important to validate behavior across all possible flows. As a result, this limitation prevented comprehensive testing and identification of all potential issues, particularly in terms of negative testing scenarios. Some of the findings reported in this assessment could have been detected if the testing suite had been more comprehensive. To ensure the robustness of the smart contracts, we recommend that Nunchi expand the testing suite to cover all functionality, including the negative testing outlined in the threat model.

To enhance the security and reliability of the protocol, we strongly recommend a thorough reaudit after development is complete and before deploying the protocol's contracts. This reaudit will help identify any potential issues or vulnerabilities, allowing for necessary changes or fixes prior to deployment.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional

guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.